

Week 2: Python Enough for Data

AI Frameworks

Today's learning goals

By the end of this lecture you should be able to:

1. Use **lists** and **dicts** for small collections of data
2. Write simple **loops** and **functions** for repeated work
3. Recognise `None`, basic errors, and a first look at reading files
4. Explain why we use **libraries** (and do not reinvent pandas)

Agenda (80 minutes)

1. Recap from Week 1
2. Lists (sequences of values)
3. Dictionaries (labelled values)
4. Loops and functions
5. Files, `None`, and errors (lightly)
6. Why libraries matter

Short quizzes check understanding—not a graded exam.

Where we are

- Week 1: AI / ML ideas, supervised learning, tools
- Today: enough Python to talk about **data** before pandas
- Next week: pandas DataFrames (tables done properly)

Remember

We only learn the Python pieces we need for data work—not every language feature.

What is a list?

A **list** stores an ordered sequence of values.

- Square brackets: [10, 20, 30]
- Indexing starts at 0
- Can hold numbers, strings, or mixed types (keep it simple for now)

Creating and indexing

```
1 scores = [80, 90, 70, 100]
2 print(scores[0])      # 80 (first)
3 print(scores[-1])     # 100 (last)
4 print(len(scores))    # 4
5
6 scores.append(85)      # add at the end
7 print(scores)
```

Slicing (gentle)

```
1 sizes = [50, 65, 80, 95, 110]
2 print(sizes[1:4])      # [65, 80, 95]  -- from index 1 up to (not
   including) 4
3 print(sizes[:2])       # [50, 65]
4 print(sizes[2:])       # [80, 95, 110]
```

You do not need every slice trick today—just know lists can give you a “chunk”.

Lists and data thinking

- One column of a table $\approx$ a list of values
- Example: all house sizes, all exam scores
- Later, pandas will hold many columns together cleanly

Common mistake

Using `scores[len(scores)]` — last valid index is `len(scores) - 1`.

Quiz 1 — Lists

```
1 xs = [3, 1, 4, 1, 5]
2 print(xs[2])
```

Quiz

What is printed?

1. 3
2. 1
3. 4
4. 5

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer — Quiz 1

Answer

3. 4 — index $0 \rightarrow 3$, $1 \rightarrow 1$, $2 \rightarrow 4$.

What is a dictionary?

A **dict** maps **keys** to **values** (like a labelled lookup).

- Curly braces: `\{"size": 70, "rooms": 3\}`
- Keys are often strings
- Great for one “row” of named fields

Creating and looking up

```
1 house = {"size": 70, "rooms": 3, "city": "yes"}
2 print(house["size"])           # 70
3 print(house.get("price", "unknown")) # safe default
4
5 house["price"] = 200           # add or update
6 print(house.keys())
```

List vs dict (when to use which)

	List	Dict
Access by	position (0, 1, 2, ...)	name / key
Good for	a column of values	one record with fields
Example	all scores	one student's info

Real tables = many rows; pandas will handle that better than nested lists of dicts.

Quiz 2 — Dicts

Quiz

You want to store one movie as title, year, and rating with clear names. Which structure fits best?

1. A list of three numbers only
2. A dictionary with keys like "title", "year", "rating"
3. A single string with no structure
4. A matplotlib colour

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

2. A dictionary — named fields are easy to read and update.

Why loops?

Repeat the same idea for every item—without copy-paste.

- Sum all scores
- Print every name
- Count how many values are above 50

for loops over lists

```
1 scores = [80, 90, 70]
2 total = 0
3 for s in scores:
4     total = total + s
5 print(total)           # 240
6 print(total / len(scores)) # mean
```

range for counting

```
1 for i in range(3):
2     print(i)      # 0 1 2
3
4 # Index + value together:
5 names = ["Ada", "Alan", "Grace"]
6 for i, name in enumerate(names):
7     print(i, name)
```

Functions: name a recipe

```
1 def mean(values):  
2     total = 0  
3     for v in values:  
4         total = total + v  
5     return total / len(values)  
6  
7 print(mean([10, 20, 30])) # 20.0
```

- Inputs = parameters; output = return
- Reuse the same logic on many lists

Good habits early

- Meaningful names: `scores`, not `x1`
- One function = one clear job
- Test with a tiny example you can check by hand

Try this

In practice: write `my_min` and `my_max` without using Python's built-ins first—then compare.

Quiz 3 — What does this print?

```
1 def double(n):  
2     return n * 2  
3  
4 xs = [1, 2, 3]  
5 out = []  
6 for x in xs:  
7     out.append(double(x))  
8 print(out)
```

Quiz

Output?

1. [1, 2, 3]
2. [2, 4, 6]
3. 6
4. An error

Answer

2. [2, 4, 6] — each element is doubled and collected in out.

Reading files (idea first)

- Data often lives in files (CSV = comma-separated values)
- Python can open a text file and read lines
- Next week pandas will load CSV in one line—today we see the idea

Tiny file read (awareness)

```
1 # Conceptual demo -- path must exist on your machine
2 with open("tiny.csv", "r") as f:
3     for line in f:
4         print(line.strip())
```

with open(...) closes the file cleanly when finished.

None means “no value”

```
1 result = None
2 if result is None:
3     print("No result yet")
4
5 # Missing data in the real world often shows up as empty / NaN later
   in pandas
```

Use `is None` (not `== None`) as the usual style check.

Errors are information

```
1 scores = [10, 20]
2 # print(scores[5])      # IndexError: list index out of range
3
4 # print(10 / 0)         # ZeroDivisionError
```

- Read the **error type** and the **line number**
- Fix one problem; re-run

Common mistake

Ignoring the traceback and randomly changing unrelated code.

Quiz 4 — None and errors (true/false)

Quiz

True or false?

`IndexError` often means you used a list index that does not exist.

1. True
2. False

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer — Quiz 4

Answer

True. Check `len(...)` and remember indexing starts at 0.

Don't reinvent pandas

Hand-written loops are great for learning—but:

- Real tables have many columns and messy types
- pandas gives `read_csv`, filters, `groupby`, ...
- matplotlib / scikit-learn solve plotting and models

Your skill: **ask the right question** and use the right tool.

Mental model for the next weeks

1. Python basics (this week) — control the computer
2. pandas — control *tables*
3. plots — see patterns
4. scikit-learn — learn patterns from labelled examples

Quiz

Why will we use pandas soon instead of only lists of dicts?

1. Because lists are illegal in Python
2. Because tables with many rows/columns are easier and safer with a dedicated library
3. Because pandas replaces the need to think
4. Because Jupyter cannot run plain Python

Pause — answer (clicker / raise hand / neighbour) before the next slide.

Answer

2. Dedicated libraries handle real tabular workflows more clearly and reliably.

Key takeaways

1. Lists = ordered values; dicts = named fields
2. Loops repeat; functions package reusable recipes
3. `None` and error messages are part of normal coding
4. Libraries (pandas next) save us from reinventing table tools

Next up

Practice (today)

Warm-ups on lists/strings · mean/min/max functions · optional tiny CSV

Next lecture (Week 3)

pandas DataFrames: load CSV, head, shape, select and filter

Which feels shakiest: lists, dicts, loops, or functions?